Докладчик

- Жаворонков Кирилл Александрович
- Студент
- Группа: НПИбд-01-23
- Номер студенческого билета: 1132231844

Цель работы

- Освоить циклы while и for, функции и сторонние пакеты Julia.
- Решать задачи линейной алгебры и работать с матрицами.
- Выполнить примеры раздела 3.2 и задания 3.4.1–3.4.11.

Слайд 1. Циклы for и двумерный массив

- Цикл for с шагом: 1, 3, 5, 7, 9.
- Перебор массива строк с приветствием.
- Двумерный массив: значение ячейки равно сумме индексов.

[3]:
```
for n in 1:2:10
println(n)
end
myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
println("Hi $friend, it's great to see you!")
end
```

```
1
3
5
7
9
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

[6]:
```
m, n = 5, 5
A = fill(0, (m, n))
# формирование массива, в котором значение каждой записи
# является суммой индексов строки и столбца:
for i in 1:m
for j in 1:n
A[i, j] = i + j
end
end
A
```

[6]:
```
5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

Слайд 2. Три способа построить массив

- fill + цикл for, цикл for i in 1:m, j in 1:n и генератор.
- Все три варианта дают один и тот же массив.
- Проверка FizzBuzz для N = 15: результат FizzBuzz.

[7]:
```
# инициализация массива m x n из нулей:
B = fill(0, (m, n))
for i in 1:m, j in 1:n
B[i, j] = i + j
end
B
```

[7]:
```
5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

[8]:
```
C = [i + j for i in 1:m, j in 1:n]
C
```

[8]:
```
5×5 Matrix{Int64}:
 2  3  4  5   6
 3  4  5  6   7
 4  5  6  7   8
 5  6  7  8   9
 6  7  8  9  10
```

[15]:
```
# используем `&&` для реализации операции "AND"
# операция % вычисляет остаток от деления
N = 15
if (N % 3 == 0) && (N % 5 == 0)
println("FizzBuzz")
elseif N % 3 == 0
println("Fizz")
elseif N % 5 == 0
println("Buzz")
else
println(N)
end
```

```
FizzBuzz
```

Слайд 3. Условия, тернарный оператор, функции

- Тернарный оператор: (5 > 10) ? 5 : 10 даёт 10.
- Функция sayhi("C-3PO") и f(42) = 1764.
- Три способа объявления функции.

```
[16]: x = 5
      y = 10
      (x > y) ? x : y

[16]: 10

[19]: function sayhi(name)
      println("Hi $name, it's great to see you!")
      end

[19]: sayhi (generic function with 1 method)

[20]: function f(x)
      x^2
      end

[20]: f (generic function with 1 method)

[21]: sayhi("C-3PO")
      f(42)

      Hi C-3PO, it's great to see you!
[21]: 1764

[22]: sayhi2(name) = println("Hi $name, it's great to see you!")
      f2(x) = x^2

[22]: f2 (generic function with 1 method)

[23]: sayhi3 = name -> println("Hi $name, it's great to see you!")
      f3 = x -> x^2

[23]: #8 (generic function with 1 method)
```

Слайд 4. sort и sort!

- sort(v) возвращает новый массив, v не меняется.
- sort!(v) сортирует массив на месте.

```
[24]: v = [3, 5, 2]
      sort(v)
      v
      sort!(v)
      v
```

```
[24]: 3-element Vector{Int64}:
       2
       3
       5
```

```
[25]: v = [3, 5, 2]
      sort(v)
      v
```

```
[25]: 3-element Vector{Int64}:
       3
       5
       2
```

```
[26]: map(f, [1, 2, 3])
```

```
[26]: 3-element Vector{Int64}:
       1
       4
       9
```

```
[27]: [f(1), f(2), f(3)]
```

```
[27]: 3-element Vector{Int64}:
       1
       4
       9
```

Слайд 5. map и broadcast

- map(f, [1, 2, 3]) даёт 1, 4, 9.
- map(x -> x^3, ...) даёт 1, 8, 27.
- broadcast и f.(...) дают тот же результат.
- Матрица A = [1 2 3; 4 5 6; 7 8 9].

```
[28]: f(x) = x^2
      map(f, [1, 2, 3])

[28]: 3-element Vector{Int64}:
       1
       4
       9

[29]: x -> x^3
      map(x -> x^3, [1, 2, 3])

[29]: 3-element Vector{Int64}:
        1
        8
       27

[30]: f(x) = x^2
      broadcast(f, [1, 2, 3])

[30]: 3-element Vector{Int64}:
       1
       4
       9

[31]: f.([1, 2, 3])

[31]: 3-element Vector{Int64}:
       1
       4
       9

[32]: A = [i + 3*j for j in 0:2, i in 1:3]

[32]: 3×3 Matrix{Int64}:
       1  2  3
       4  5  6
       7  8  9
```

Слайд 6. Пакет Colors

- Палитра distinguishable_colors(100).
- rand(palette, 3, 3) — случайная матрица 3×3 цветов.



Слайд 7. Задания 3.4.2–3.4.4

- 3.4.2: if и тернарный оператор; для 14 — число, для 15 — «нечётное».
- 3.4.3: add_one(5) = 6.
- 3.4.4: матрица [1 4 7; 2 5 8; 3 6 9] через map.

[46]:
```
num = 14
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
14
```

[47]:
```
num = 15
if num % 2 == 0
    println(num)
else
    println("нечётное")
end
```

```
нечётное
```

[48]:
```
num % 2 == 0 ? println(num) : println("нечётное")
```

```
нечётное
```

[49]:
```
add_one(x) = x + 1
println("add_one(5) = ", add_one(5))
```

```
add_one(5) = 6
```

[50]:
```
A_task4 = reshape(map(add_one, 0:8), 3, 3)
println("Матрица A (каждый элемент на 1 больше): \n", A_task4)
```

```
Матрица A (каждый элемент на 1 больше):
[1 4 7; 2 5 8; 3 6 9]
```

Слайд 8. Задания 3.4.5–3.4.7

- 3.4.5: A^3 — нулевая матрица; третий столбец заменён суммой второго и третьего.
- 3.4.6: $C = B^T B$ = [1500 −1500 1500; −1500 1500 −1500; 1500 −1500 1500].
- 3.4.7: матрица Z1 совпадает с эталоном.

[51]:
```
A_5 = [1  1  3;
       5  2  6;
      -2 -1 -3]

A_cubed = A_5^3
println("A^3 = \n", A_cubed)

A_5[:, 3] = A_5[:, 2] + A_5[:, 3]
println(A_5)
```

```
A^3 =
[0 0 0; 0 0 0; 0 0 0]
[1 1 4; 5 2 8; -2 -1 -4]
```

[52]:
```
B = fill(10, 15, 3)
B[:, 2] .= -10
C_mat = B' * B
println("Матрица C = B^T * B: \n", C_mat)
```

```
Матрица C = B^T * B:
[1500 -1500 1500; -1500 1500 -1500; 1500 -1500 1500]
```

[53]:
```
Z = zeros(Int, 6, 6)
E = ones(Int, 6, 6)

Z1 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    if abs(i - j) == 1
        Z1[i, j] = 1
    end
end
println("Матрица Z1:\n", Z1)
```

```
Матрица Z1:
[0 1 0 0 0 0; 1 0 1 0 0 0; 0 1 0 1 0 0; 0 0 1 0 1 0; 0 0 0 1 0 1; 0 0 0 0 1
0]
```

Слайд 9. Функция outer (3.4.8)

- outer(x, y, operation) строит матрицы поэлементно.
- A1, A3, A4 совпадают с эталоном.
- A2 и A5 требуют исправления; обобщение на матричное умножение не выполнено.

[56]:

```
function outer(x, y, operation)
    return [operation(i, j) for i in x, j in y]
end

A1 = outer(0:4, 0:4, +)
println("A1:\n", A1)

A2 = outer(0:4, 1:5, (x, y) -> x == 0 ? 0 : x^(y-1))
println("\nA2:\n", A2)

A3 = outer(0:4, 0:4, (x, y) -> (x + y) % 5)
println("\nA3:\n", A3)

A4 = outer(0:9, 0:9, (x, y) -> (x + y) % 10)
println("\nA4:\n", A4)

A5 = outer(1:10, 1:10, (x, y) -> (x - y + 10) % 10)
println("\nA5:\n", A5)
```

```
A1:
[0 1 2 3 4; 1 2 3 4 5; 2 3 4 5 6; 3 4 5 6 7; 4 5 6 7 8]

A2:
[0 0 0 0 0; 1 1 1 1 1; 1 2 4 8 16; 1 3 9 27 81; 1 4 16 64 256]

A3:
[0 1 2 3 4; 1 2 3 4 0; 2 3 4 0 1; 3 4 0 1 2; 4 0 1 2 3]

A4:
[0 1 2 3 4 5 6 7 8 9; 1 2 3 4 5 6 7 8 9 0; 2 3 4 5 6 7 8 9 0 1; 3 4 5 6 7 8
9 0 1 2; 4 5 6 7 8 9 0 1 2 3; 5 6 7 8 9 0 1 2 3 4; 6 7 8 9 0 1 2 3 4 5; 7 8
9 0 1 2 3 4 5 6; 8 9 0 1 2 3 4 5 6 7; 9 0 1 2 3 4 5 6 7 8]

A5:
[0 9 8 7 6 5 4 3 2 1; 1 0 9 8 7 6 5 4 3 2; 2 1 0 9 8 7 6 5 4 3; 3 2 1 0 9 8
7 6 5 4; 4 3 2 1 0 9 8 7 6 5; 5 4 3 2 1 0 9 8 7 6; 6 5 4 3 2 1 0 9 8 7; 7 6
5 4 3 2 1 0 9 8; 8 7 6 5 4 3 2 1 0 9; 9 8 7 6 5 4 3 2 1 0]
```

Слайд 10. СЛАУ и случайная матрица (3.4.9–3.4.10)

- 3.4.9: x ≈ [−2, 3, 5, 2, −4].
- 3.4.10: для одного запуска — элементов больше 4 по строкам: [9, 8, 8, 8, 3, 6]; строк с 7 ровно дважды нет.

```
5 4 3 2 1 0 9 8; 8 7 6 5 4 3 2 1 0 9; 9 8 7 6 5 4 3 2 1 0]
```

[57]:
```
n_eq = 5
A_sys = [abs(i - j) + 1 for i in 1:n_eq, j in 1:n_eq]
y_sys = [7, -1, -3, 5, 17]
x_sys = A_sys \ y_sys
println(x_sys)
```

```
[-1.9999999999999987, 2.9999999999999996, 4.999999999999998, 2.0000000000000001, -4.0]
```

[58]:
```
M = rand(1:10, 6, 10)
println(M)

N_val = 4
gt_N_counts = [count(>(N_val), M[i, :]) for i in 1:size(M, 1)]
println(gt_N_counts)

M_val = 7
rows_with_two_7 = findall(i -> count(==(M_val), M[i, :]) == 2, 1:size(M, 1)
println(rows_with_two_7)

K_val = 75
col_pairs = []
num_cols = size(M, 2)
for j1 in 1:num_cols
    for j2 in (j1+1):num_cols
        if sum(M[:, j1]) + sum(M[:, j2]) > K_val
            push!(col_pairs, (j1, j2))
        end
    end
end
println(col_pairs)
```

```
[10 10 5 8 10 2 6 5 6 10; 7 7 10 7 7 10 7 5 2 1; 6 9 10 8 5 4 1 8 9 10; 9 6 2 7 5 10 1 8 5 5; 3 1 7 3 4 5 3 6 2 4; 4 1 8 6 10 2 3 9 6 9]
[9, 8, 8, 8, 3, 6]
Int64[]
Any[(1, 3), (1, 4), (1, 5), (1, 8), (1, 10), (2, 3), (3, 4), (3, 5), (3, 8), (3, 10), (4, 5), (4, 8), (4, 10), (5, 8), (5, 10), (8, 10)]
```

Слайд 11. Двойные суммы (3.4.11)

- Сумма $i^4/(3+j) \approx 639215.2833$.
- Сумма $i^4/(3+i \cdot j) \approx 89912.0215$.

```
8), (3, 10), (4, 5), (4, 8), (4, 10), (5, 8), (5, 10), (8, 10)]

[59]: sum_1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
println(sum_1)

sum_2 = sum(i^4 / (3 + i * j) for i in 1:20, j in 1:5)
println(sum_2)

639215.2833333338
89912.02146097131

[ ]:
```

Ограничения работы

- Не предоставлены скриншоты 3.4.1 (квадраты, словарь, массив) и Z2–Z4 из 3.4.7.
- Матрица M в 3.4.10 не зафиксирована генератором случайных чисел.
- В 3.4.8 требуется исправить A2 и A5 и обобщить outer на матричное умножение.

Выводы

- Освоены циклы while и for, условные и тернарный операторы, функции.
- Освоены map, broadcast, матричные операции и решение СЛАУ.
- Подключены пакеты Colors и LinearAlgebra.
- Отмечены особенности Julia: A^3 и A.^3 различаются; sort! меняет массив.